Distilled Command Parser on Local Kubernetes

Cloud Computing Assignment Report | Data pipeline, network controls, model distillation and resilient inference

Student	Roll number	Submission date
Arihant Kumar	230101028	9 November 2026

Executive summary

This project implements an end-to-end command parsing system on an 8 GB Apple M1 machine. A Phi-3.5 teacher labelled 3,000 supplied commands in two deterministic Kubernetes jobs. Validated records were stored in an S3-compatible SeaweedFS service and used to distil EleutherAI/Pythia-70M. The selected v2 student was packaged behind a strict FastAPI contract, deployed to k3d, autoscaled under Locust traffic, and tested for pod loss and malformed model output.

SCOPE All numerical claims marked as measured come from committed files in results/. Cost uses the supplied rate card and is explicitly an estimate, not a cloud invoice.

End-to-end architecture

Command pool -> **Phi-3.5 jobs** -> **SeaweedFS** -> **Pythia-70M** -> **FastAPI** -> **k3d + HPA**

Headline measured outcomes

Stage	Result	Status
Teacher data	2,798 clean records from 3,000 inputs	Complete
Selected student v2	99% JSON validity; 79% intent; 46% full frame	Quality gate PASS
Optimized service	95.12% success; 8.1 s median; 26 s p95	SLO PASS
Resilience	8 s pod recovery; 25/25 health probes HTTP 200	SLO PASS
Release	10 assets with server-side SHA-256 verification	Published

Repository: github.com/kumararihant-06/Cloud_Assignment_1028
Model release: releases/tag/model-v2

TASK 1 Distributed teacher labelling and object storage

Deterministic generation, complete shard coverage, cleaning and provenance

Architecture

The supplied commands_pool.jsonl is uploaded to a SeaweedFS S3 bucket. Two Kubernetes Jobs partition work by numeric ID modulo two and call a locally hosted Phi-3.5 teacher. Each job writes an independent JSONL shard. A separate cleaning Job validates the fixed schema, removes invalid slot values and duplicate commands, balances output deterministically, and writes the training dataset plus a checksum back to object storage.



Implementation

- Teacher generation uses temperature 0, seed 42, a fixed prompt, bounded retries and no access to validation labels.
- Kubernetes Secrets provide S3 credentials at runtime; credentials are generated locally and never committed.
- Object keys separate raw teacher shards from datasets/v1/train.jsonl; exports preserve logs and cluster state under results/.
- The cleaning algorithm keeps the smallest source ID for duplicate text, making reruns reproducible.

Measured results

Measure	Observed value
Shard coverage	1,500 + 1,500; 3,000 unique IDs; none missing or unexpected
Teacher job duration	Approximately 73 minutes per parallel worker
Rejected records	153 invalid slot values; 49 duplicate commands
Clean records	2,798 of 3,000 (93.27%)
Dataset digest	1331d0275bd00ee6066eccecf6d2e2c492dd1a51a522563d717ec3fbfeb5f2d8

Limitations

- Syntactic validation cannot prove semantic correctness; teacher evaluation later measured only 40% exact full-frame accuracy on 100 labelled examples.
- The experiment used one local k3d node and local Ollama, so it does not measure multi-node scheduling, network transfer or managed S3 behaviour.

Improvements

- Add a human-reviewed semantic audit sample, per-intent error counts and an explicit low-confidence review queue.
- Use checkpointed/resumable shard writes and multiple worker replicas when a multi-node cluster is available.

EVIDENCE results/task1_shard_coverage.json, results/task1_cleaning_report.json, worker/, scripts/export_task1_evidence.py and kubernetes/teacher-workers.yaml

TASK 2 VLSM subnetting and stateless network ACL

Capacity-aware address allocation with ordered allow and deny rules

Architecture

The 10.20.0.0/16 address space is allocated largest-first using variable-length subnet masks. A public-subnet network ACL enforces default deny, permits web ingress, restricts SSH to the management subnet, permits application traffic on TCP/8000, and explicitly allows ephemeral reply ports because network ACLs are stateless.

Implementation and address plan

Subnet	Need	CIDR	Usable range	Capacity
Application	1,000	10.20.0.0/22	10.20.0.1 - 10.20.3.254	1,022
Public	250	10.20.4.0/24	10.20.4.1 - 10.20.4.254	254
Data	100	10.20.5.0/25	10.20.5.1 - 10.20.5.126	126
Management	20	10.20.5.128/27	10.20.5.129 - 10.20.5.158	30

- The deny for abusive source 198.51.100.0/24 is rule 50, before the general HTTP/HTTPS allows at rules 100 and 110.
- Inbound TCP/22 accepts only 10.20.5.128/27. Inbound replies from the application tier and outbound client replies use TCP 1024-65535.
- Outbound TCP/8000 targets only the application subnet. Any unmatched packet is denied.

Measured results

6 / 6 PASS	The supplied packet cases were evaluated by tools/nacl_sim.py. Web access, abusive-source precedence, management-only SSH, application forwarding, ephemeral replies and default-deny behaviour matched expectations.

Limitations

- The ACL was validated with deterministic packet cases, not deployed to a public cloud VPC; provider-specific reserved-address and rule-limit behaviour was not exercised.
- The design covers IPv4 and TCP only and does not replace host firewalls, security groups, TLS policy or application authentication.

Improvements

- Express the plan in Terraform and validate it in an ephemeral cloud account with flow logs and automated reachability tests.
- Add IPv6, explicit DNS/NTP requirements, egress filtering and a stateful security-group layer.

EVIDENCE	network/subnets.md, network/nacl.json, network/packets_sample.jsonl, tools/nacl_sim.py and tests/test_network.py

TASK 3 CPU model distillation and evaluation

A reproducible Pythia-70M student selected by held-out quality evidence

Architecture

The clean teacher dataset is converted into prompt/target sequences for EleutherAI/Pythia-70M. Loss is masked on prompt tokens so optimization learns only the JSON target. Training runs on CPU with fixed seeds and writes model, tokenizer, loss history, package versions and SHA-256 manifests to SeaweedFS. Four systems are evaluated on the same 100-example labelled validation set.

Implementation

- Shared configuration: 1,500 training examples, batch size 4, learning rate 5e-5, maximum sequence length 192 and gradient clipping at 1.0.
- v1 uses seed 42 for 3 epochs. v2 changes only seed to 43 and trains for 4 epochs, preserving a controlled comparison.
- The parser enforces exactly five keys: intent, date, time, place and no additional fields. A 95% JSON-validity gate selects release candidates.

Measured training results

Run	CPU wall time	Epoch losses	Final model SHA-256
v1	~6 min 42 s	0.3885, 0.1970, 0.1547	3606a3fd...3a6f11a
v2	~5 min 14 s	0.3822, 0.2053, 0.1693, 0.1378	a1fe3929...b92e8c

Measured evaluation on 100 examples

System	JSON valid	Intent	Date	Time	Place	Full frame
Untrained Pythia	0%	0%	0%	0%	0%	0%
Phi-3.5 teacher	100%	79%	77%	85%	76%	40%
Student v1	97%	76%	75%	79%	77%	36%
Student v2	99%	79%	80%	89%	83%	46%

SELECTION v2 passed the 95% validity gate, tied the teacher on intent accuracy and achieved the best slot and full-frame scores.

Limitations

- The validation set contains only 100 examples; full-frame accuracy of 46% remains insufficient for high-stakes automation.
- Student quality inherits teacher noise, and only 1,500 of 2,798 clean examples were used to keep CPU training practical.

Improvements

- Use all clean data, a larger stratified gold set, constrained JSON decoding, error-driven augmentation and confidence calibration.
- Benchmark quantization and compact classification/slot models to reduce latency while retaining schema validity.

TASK 4 Containerized inference and Kubernetes operations

Strict API contracts, verified model loading, autoscaling and defensive deployment

Architecture



At pod startup, the selected v2 artifact is downloaded from S3-compatible storage and verified against model.sha256. Readiness stays false until the tokenizer and model load. FastAPI validates requests, normalizes model output and rejects malformed or extra-key JSON with a controlled HTTP 500. Kubernetes exposes the workload through a ClusterIP Service and Traefik ingress.

Implementation

Concern	Implemented control
API contract	POST /parse; strict request and five-key response schemas; model version and serving pod headers
Health	Separate /healthz and /readyz endpoints with startup, liveness and readiness probes
Scaling	HPA minimum 2, maximum 4, target 60% average CPU; 500m CPU request per pod
Resources	2 CPU and 1 GiB limits; 512 MiB memory request; four inference threads
Availability	Rolling updates, topology spread and PodDisruptionBudget minAvailable 1
Container security	UID 10001, read-only root filesystem, no privilege escalation and all Linux capabilities dropped
Supply integrity	Model files excluded from Git; startup checksum validation; GitHub Release checksum manifest

Test strategy

- Unit tests cover cleaning, training masks/configuration, strict schema handling, API behaviour, cost arithmetic and manifest constraints.
- Integration evidence includes image build, Kubernetes rollout state, HTTP response headers, Locust history, HPA state and fault injection.
- GitHub Actions runs tests, selected-model quality gate, NACL cases, cost recalculation, container build and a full-history Gitleaks scan.

Operational flow

Build	Deploy	Observe	Shutdown
docker build / make service-image	make service-deploy	make service-status; curl; Locust	make service-down; make down

DESIGN CHOICE A two-replica minimum preserves service during one-pod loss. HPA reaches four replicas under measured CPU saturation, while the PDB protects voluntary disruptions.

TASK 4 Measured performance, resilience and cost

Baseline-to-optimized comparison and explicit operating assumptions

Measured Locust results: 4 users, spawn rate 4/s, 2 minutes

Metric	1 CPU baseline	2 CPU optimized	Change
Requests	17	41	+24
Success rate	41.18%	95.12%	+53.94 percentage points
Throughput	0.1451 req/s	0.3437 req/s	2.37x
Average latency	25.66 s	10.65 s	-58.50%
Median latency	30.0 s	8.1 s	-73.00%
p95 latency	30.0 s	26.0 s	-13.33%

The baseline saturated one CPU per pod and timed out during HPA scale-up. A controlled 45-second probe at a two-CPU limit produced 8/8 successes, motivating the final setting. In the complete optimized run, 39/41 requests succeeded; the remaining two timeouts occurred while HPA moved from two to four ready replicas.

Measured resilience

Experiment	Observed result
Single-pod deletion	Replacement ready in 8 seconds; 25/25 ingress health probes returned HTTP 200
Malformed model output	Injected response HTTP 500 with {error: invalid_model_output}; restored response HTTP 200
Model traceability	39/39 successful load responses reported v2 and were distributed across four serving pods

Cost estimate and SLO status

Item	Value	Classification
Four-replica cost	INR 29.80/hour	Assumption from supplied rate card
Cost per 1,000 attempted	INR 24.0843	Estimate using measured throughput
Cost per 1,000 successful	INR 25.3194	Estimate using measured success ratio
Quality / success / p95 / recovery SLOs	99% / 95.12% / 26 s / 8 s	Measured; all PASS

Limitations

- The two-minute local test is not a production endurance test. Latency includes CPU generation on an M1 Docker/k3d environment and cannot be generalized to cloud hardware.
- Cold HPA reconciliation caused two timeouts. The cost model assumes four replicas remain active and bills declared limits continuously; no cloud invoice was observed.

Improvements

- Pre-warm four replicas for burst windows, tune HPA behaviour, add a queue with backpressure, and test longer step-load and soak scenarios.
- Quantize or replace the generative parser with a smaller structured model; add OpenTelemetry, dashboards, alerts and canary rollback.

VALIDATION Reproducibility, security and provenance

Every deliverable is linked to executable code, immutable evidence or a checksum

Verification layers

Layer	Verification
Source	Git history with student-controlled commits; final code organized by data, worker, training, evaluation, service, Kubernetes, network and tools
Automated tests	29 local unit/manifest tests passed; CI run 2 completed successfully on commit 88a94f4
Security	Gitleaks full-history scan passed; generated Secrets and model weights are excluded from Git
Data	Complete 3,000-ID shard coverage and immutable clean-dataset SHA-256
Model	SeaweedFS manifests verified; GitHub server digest equals local selected-model digest
Deployment	Captured rollout, HPA, headers, Locust, pod-loss and injected-failure evidence

Release provenance

The public model-v2 GitHub Release targets commit 88a94f497f98edd1a97d6ff761ccb07c79c72d0a. It contains ten assets: model weights, tokenizer files, configurations, loss history, version metadata, the training checksum and a release-wide SHA256SUMS file. The 281,715,176-byte model digest is a1fe3929360f76faa8f769219b4479919305f710a7155c0dafa835ba86b92e8c.

Measured facts versus assumptions

Measured directly	Assumed or project-defined
Data counts, checksums, training loss/time and 100-example evaluation	The 100-example sample represents future traffic
Two Locust runs, response headers, HPA observations and CPU limits	Four replicas are a suitable steady-state costing point
Pod replacement time, health responses and malformed-output behaviour	Local-cluster resilience predicts multi-node cloud behaviour
CI job conclusions and GitHub asset digests	Supplied rate-card prices represent deployable cloud cost

Reproduction sequence

1. make cluster-up storage-bootstrap 2. start Ollama/Phi-3.5 3. make teacher-jobs clean-data task1-evidence 4. train/evaluate v1 and v2 5. make service-deploy load-test cost-analysis 6. run tests, scan, release checksums and shutdown.

AI DISCLOSURE OpenAI Codex assisted planning, coding and diagnosis. The student reviewed changes, executed every commit/push and retained responsibility for design and verification. AI suggestions are not used as evidence; see AI_USE.md.

CONCLUSION Findings and next steps

A complete local cloud-native ML system with transparent evidence and known limits

Key findings

- Deterministic sharding and checksum-backed object storage made a long teacher-labelling stage auditable and repeatable.
- The 70M student matched teacher intent accuracy and improved exact full-frame accuracy from 40% to 46%, but semantic quality remains the main system constraint.
- CPU allocation dominated inference reliability: raising the limit from one to two CPUs reduced failures from 58.82% to 4.88% and increased throughput 2.37x.
- Replica redundancy and a PDB prevented observed health downtime during one-pod loss, while strict schema handling converted malformed output into a controlled error.

Prioritized improvements

Priority	Improvement	Expected effect
1	Constrained decoding plus larger human-reviewed validation set	Higher semantic and full-frame confidence
2	Quantized or discriminative parser	Lower latency and cost on CPU
3	HPA pre-warming, tuned behaviour and request queue	Remove burst-window timeouts
4	Multi-node cloud test, flow logs and IaC	Validate failure-domain and network assumptions
5	Soak tests, tracing, dashboards and alerts	Operational confidence beyond short local runs

References

[1] Assignment-supplied command pool, validation labels, schema, packet cases and rate card, 2026.

[2] EleutherAI, Pythia-70M model card. https://huggingface.co/EleutherAI/pythia-70m

[3] Microsoft, Phi-3.5 Mini Instruct model card. https://huggingface.co/microsoft/Phi-3.5-mini-instruct

[4] SeaweedFS documentation. https://github.com/seaweedfs/seaweedfs

[5] Kubernetes, Horizontal Pod Autoscaling. https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/

[6] Kubernetes, Configure liveness, readiness and startup probes. https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/

[7] FastAPI documentation. https://fastapi.tiangolo.com/

Evidence index

Primary measurements are in results/evaluation_summary.json, training_v1.txt, training_v2.txt, loadtest_baseline_summary.txt, loadtest_optimized_summary.txt, reliability.txt, cost.json, slo.json, ci.txt and release.txt. Build, deploy, test and shutdown commands are documented in README.md.

Final assessment: the project satisfies the required data, networking, training, deployment, testing, evidence and release workflow. Its measured limitations are reported rather than hidden, and the repository provides a reproducible path from source data to shutdown.